Ablation Study of CBMC Harness Generation System

Utsav Negi

Table of Content

1. System Overview

The CBMC Harness Generation System is an LLM-based system which generates harness functions for a given source code. These harness functions serve as a proof to verify whether a given function is free of memory and arithmetic vulnerability.

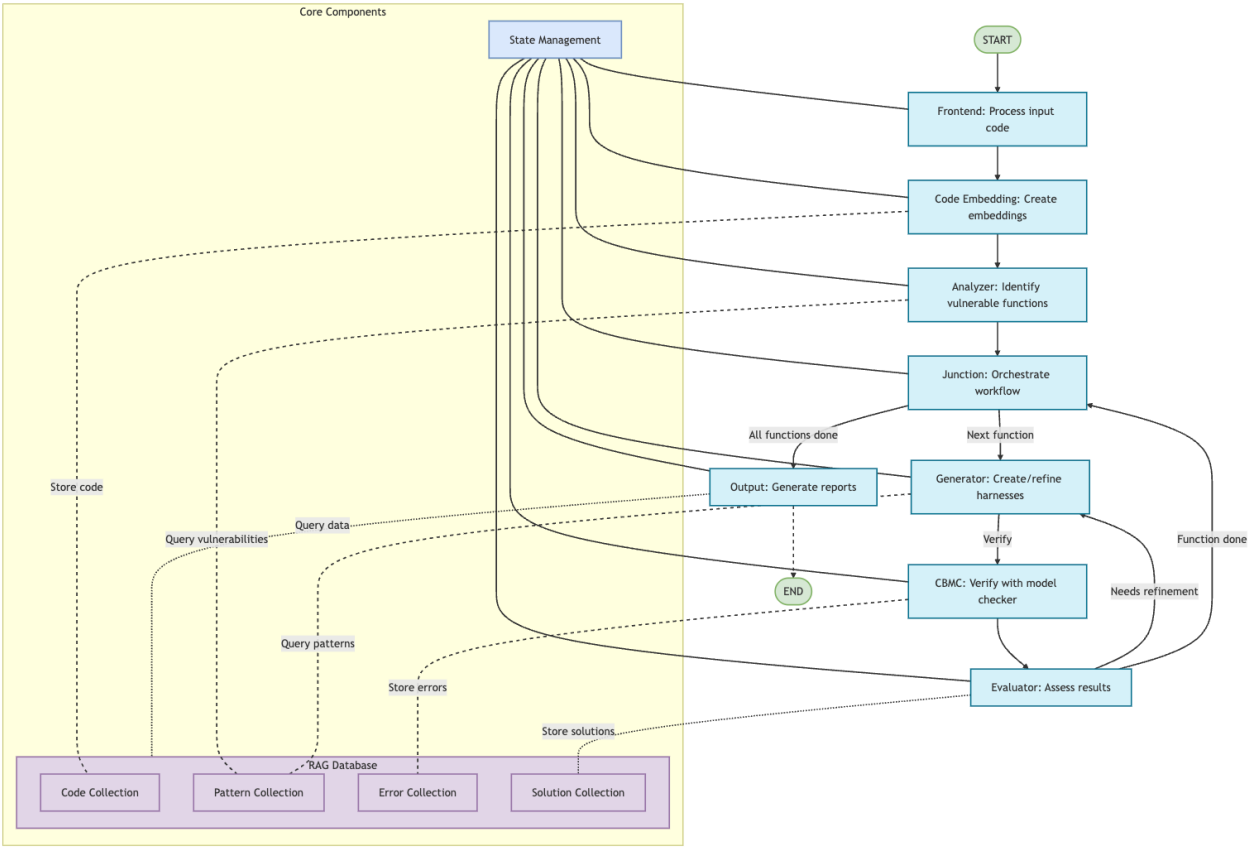


Figure 1: System Architecture

1.1 System Components

The system is composed of following components which is orchestrated using LangGraph framework

- **Frontend**
  The frontend accepts the incoming source code and its dependencies into the system. It also initializes the necessary components such as state manager, and the RAG database.

- **Code Embedding**
  This component embeds all the headers, functions, and macros into SQLite database using a transformer model.

- **Analyzer**
  The analyzer uses regular expressions to detect functions that uses memory or arithmetic operations. The system then generates a harness for these detected/target functions.

- **Junction**
  The junction component orchestrates the sequence of harness generation. If a generated harness function requires refinement, then the junction reuses the same target functions; otherwise, the junction will move on to the next target function in the list.

- **Generator**
  The generator component uses LLM model to generate harness function for a given target function using the source code knowledge stored in the RAG, and the prompts consisting of rules, and harness function template.

- **CBMC**
  This component runs the generated harness function on a C-Bounded Model Checker (CBMC) program and stores the generated stderr and stdout from the CBMC to verify the generated harness. These outputs are further processed to determine the errors and the coverage of the generated harness function.

- **Evaluator**
  The evaluator component evaluates the outputs, errors and the coverage to suggest improvement recommendation based on the type of the error or current coverage. If a generated harness function has zero errors and satisfactory coverage, then the evaluator will recommend the junction to move to the next target function, else send an improvement recommendation to the generator.

- **Output**
  After going through all the target functions, the output component collects all the error and coverage statistics and create a comprehensive report showing the number of errors and coverage percentage across each refinement stage.

- **RAG System**
  The Retrieval-Augmented Generation (RAG) System is a database-powered system to assist the generator and the evaluator component to understand the source code

by embedding the source code knowledge into a database using a transformer model. Further, the RAG system also stores knowledge about common memory-based issues, patterns, and ways to resolve those issues. Besides source code knowledge and error trends, there are dedicated databases to store the errors faced by the system during harness generation process along with the working solutions. Such application of the RAG system ensures generation of better-quality harness functions with a smaller number of refinements.

- **State Manager**
  The state manager is a property of LangGraph framework to store objects across all the components. This property allows the system to track the harness history, target functions, number of refinements done, errors, file paths, and system timing.

1.2 System Flow

The system is built as a command line tool which takes in directory path of the source code, the type of LLM for generator and evaluator component, and SAT solver for CBMC as the main arguments. The system first compiles the code to check for any errors in the source code. The system then parses through the source code and store code in the RAG databases. This is followed by determining the functions which uses memory or arithmetic operations. For each target function, there is an iteration cycle between the generator to generate the harness, CBMC to determine the performance of the harness, and the evaluator to critique the harness. This iteration is completed when a generated harness function satisfies all the desired metrics (error and coverage), or the number of maximum iterations has been reached. Finally, the output phase compiles all the statistics regarding the entire process and present it in the form of a report.

2. About this Study

The aim of this study is to observe the importance of each important component of the system. The study will cover the impact of RAG system on the overall system, the performance of system across different SAT solvers and embedding models, and the performance of the system with different kind of prompt techniques. Furthermore, the study also observes the changes made by generator to the harness functions to reduce the errors and improve the coverage. Note that ChatGPT 4.1 is used across all the LLM nodes in the orchestration with the temperature set to 0.45. For this study, two examples are used to study the system and analyze its performance. One is single file toy example while the other is a production-level code involving multiple header files and dependencies.

2.1 Example 1: Sorting Algorithm

Sorting Algorithms involve a lot of memory-based operations to read and write into an array while having arithmetic operations to compare two values. For this study, the Bubble Sort Algorithm is considered as a toy example which sort an array of numbers in increasing order. This setup involving multiple memory management and comparison mechanisms aids in analyzing the quality of harnesses that the system can generate.

2.2 Example 2: Open-Source Code

To test the performance of the system on a production-level code, the study relies on an open-source Git repository maintained by Amazon Web Services. The objective of this repository is to send IoT device metrics to the AWS IoT Device Defender Services. The main source code consists of eight functions involving either memory-based or arithmetic-based operations. Furthermore, the code has already gone through verification checks while ensuring the code is up to the MISRA coding standard.

3. Error Detection in Source Code

Before initiating the harness generation process, the system must verify that the user-provided source code is free of errors. If any errors are detected, the system promptly informs the user about them. It's important to note that warnings are not considered critical errors and are not reported by the system. Instead, the system focuses on identifying and reporting the most significant errors that disrupt the programming logic of the source code.

3.1 Detecting Error in a bugged Sorting Algorithm

When the code for the bubble sort algorithm contains an error, the system promptly detects the bug and halts the workflow. It then advises the user to resolve the error before proceeding with the harness generation.

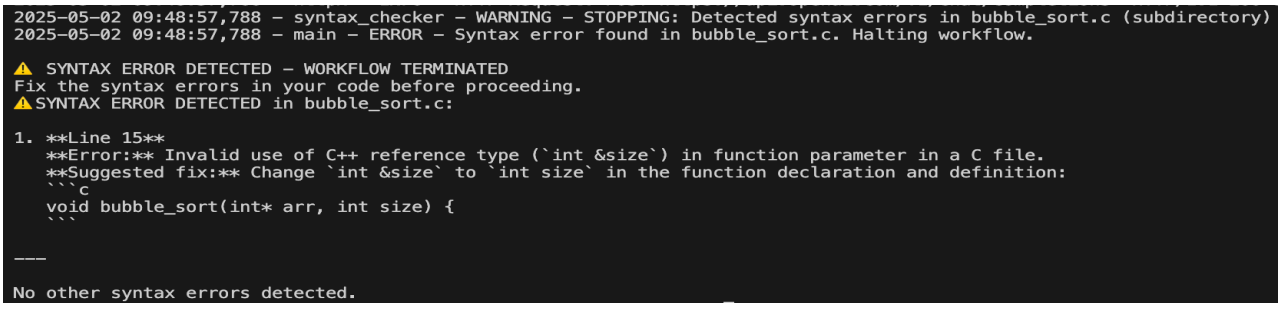


Figure 2: Terminal Output for Error Detection in Single File Example

3.2 Detecting Error in a bugged Open-Source Code

The system goes through all the header files and the c files in the source code to check for any errors in the system. If there is a single error in any one file, then the workflow is halted, and the system recommends users to fix the error. Note that in this case, the system won't explicitly specify the location of the error within the file, it only shows the files that have syntax errors.

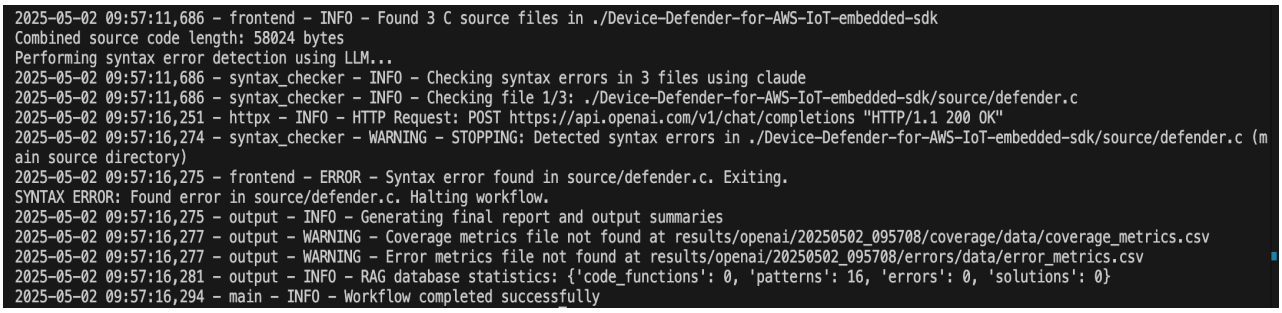


Figure 3: Terminal Output for Error Detection in Open-Source Code

4. RAG System and SAT Solvers

The RAG System plays an important role in storing the knowledge about the source code, common memory problems, errors occurred during the harness generation process and their respective solutions at one place. The properties of the RAG system has been discussed in Section 1.1. Besides, the SAT solver functions as CBMC's core problem-solving engine, tackling complex verification challenges through a streamlined process. It receives your program converted into an enormous logical formula (essentially a sophisticated true/false puzzle) and methodically searches for any possible scenario where your code might fail or violate its specifications. Using highly optimized algorithms, it efficiently navigates through billions of potential program states and variable combinations-a task that would be impossible to perform manually. If the solver discovers any execution path that could trigger an error, it provides a precise counterexample showing exactly how the failure occurs, including the specific inputs and conditions that lead to the problem. This powerful capability transforms the abstract challenge of program verification into a concrete, solvable problem, allowing developers to identify and fix bugs before they manifest in real-world situations. It is interesting to understand the performance of the system without the RAG system, and its performance with different embedding models while using different SAT solvers.

4.1 Performance without RAG using MiniSAT Solver

For the single file bubble sort example, the no RAG approach with MiniSAT solver had the following statistics:

Total execution time: 36.30 seconds
Average harness generation time: 2.93 seconds
Average verification time: 0.35 seconds
Average evaluation time: 0.02 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1

Table 1: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for a single file code as the system is running without RAG while the CBMC is using MiniSAT Solver

The system is not compatible with large open-source code with multiple header files without RAG. This is because single-file code can be easily substituted as part of the prompts, while code involving multiple header files and dependencies impacts the input token size, hindering further refinements due to the lack of a context window.

4.2 Performance without RAG using CaDiCal Solver

For the single file bubble sort example, the no RAG approach with MiniSAT solver had the following statistics:

Total execution time: 62.91 seconds
Average harness generation time: 3.76 seconds
Average verification time: 0.33 seconds
Average evaluation time: 0.01 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	4	100.00%	100.00%	4	100.00%	100.00%	0	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	1	94.12%	85.71%	1	100.00%	100.00%	1

Table 2: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for a single file code as the system is running without RAG while the CBMC is using CaDiCal Solver

The system is not compatible with large open-source code with multiple header files without RAG. This is because single-file code can be easily substituted as part of the prompts, while code involving multiple header files and dependencies impacts the input token size, hindering further refinements due to the lack of a context window.

4.3 Performance with all-MiniLM-L6-v2 Embedding Model and MiniSAT Solver

For the single file bubble sort example, the RAG approach with all-MiniLM-L6-v2 embedding model and MiniSAT Solver had the following performance

Total execution time: 46.40 seconds
Average harness generation time: 3.80 seconds
Average verification time: 0.38 seconds
Average evaluation time: 0.01 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	4	100.00%	100.00%	0	-	-	-	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1

Table 3: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for a single file code as the system is running with RAG on all-MiniLM-L6-v2 Embedding Model while the CBMC is using MiniSAT Solver

For the large open-source code example, the RAG approach with all-MiniLM-L6-v2 embedding model and MiniSAT Solver had the following performance

Total execution time: 46.12 seconds

Average harness generation time: 4.97 seconds
Average verification time: 0.45 seconds
Average evaluation time: 0.01 seconds

Function	File	Status	Version 1		
			Total %	Func %	Errors
Defender_GetTopic	defender.c	SUCCESS	100.00%	100.00%	0
Defender_MatchTopic	defender.c	SUCCESS	97.18%	95.74%	0
extractThingNameLength	defender.c	SUCCESS	100.00%	100.00%	0
getTopicLength	defender.c	SUCCESS	92.59%	90.48%	0
matchApi	defender.c	SUCCESS	100.00%	100.00%	0
matchBridge	defender.c	SUCCESS	100.00%	100.00%	0
matchPrefix	defender.c	SUCCESS	100.00%	100.00%	0
writeFormatAndSuffix	defender.c	SUCCESS	100.00%	100.00%	0

Table 4: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for the open-source code as the system is running with RAG on all-MiniLM-L6-v2 Embedding Model while the CBMC is using MiniSAT Solver

4.4 Performance with all-MiniLM-L6-v2 Embedding Model and CaDiCal Solver

For the single file bubble sort example, the RAG approach with all-MiniLM-L6-v2 embedding model and CaDiCal Solver had the following performance

Total execution time: 67.73 seconds
Average harness generation time: 4.34 seconds
Average verification time: 0.33 seconds
Average evaluation time: 0.01 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	4	100.00%	100.00%	0	-	-	-	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	2	100.00%	100.00%	2	100.00%	100.00%	1

Table 5: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for a single file code as the system is running with RAG on all-MiniLM-L6-v2 Embedding Model while the CBMC is using CaDiCal Solver

For the large open-source code example, the RAG approach with all-MiniLM-L6-v2 embedding model and CaDiCal Solver had the following performance

Total execution time: 245.01 seconds
Average harness generation time: 5.63 seconds
Average verification time: 0.38 seconds
Average evaluation time: 0.01 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4			Version 5			Version 6		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
Defender_GetTopic	defender.c	FAILED	100.00%	100.00%	1	95.92%	91.30%	1	96.00%	91.30%	1	96.00%	91.30%	1	96.00%	91.30%	1	95.65%	91.30%	1
Defender_MatchTopic	defender.c	FAILED	97.18%	95.74%	1	97.26%	95.74%	1	97.40%	95.74%	1	97.30%	95.74%	1	97.01%	95.74%	1	97.18%	95.74%	1
extractThingNameLength	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
getTopicLength	defender.c	SUCCESS	93.33%	90.48%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
matchApi	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-
matchBridge	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
matchPrefix	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-

Table 6: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for the open-source code as the system is running with RAG on all-MiniLM-L6-v2 Embedding Model while the CBMC is using CaDiCal Solver

4.5 Performance with text-embedding-ada-002 for OpenAI and MiniSAT Solver

For the single file bubble sort example, the RAG approach with text-embedding-ada-002 model and MiniSAT Solver had the following performance

Total execution time: 52.11 seconds
Average harness generation time: 2.76 seconds
Average verification time: 0.33 seconds
Average evaluation time: 0.50 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	1	96.55%	85.71%	1	100.00%	100.00%	1

Table 7: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for a single file code as the system is running with RAG on text-embedding-ada-002 Embedding Model while the CBMC is using MiniSAT Solver

For the large open-source code example, the RAG approach with text-embedding-ada-002 model embedding model and MiniSAT Solver had the following performance

Total execution time: 206.31 seconds
Average harness generation time: 6.09 seconds
Average verification time: 0.36 seconds
Average evaluation time: 0.37 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4			Version 5			Version 6		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
Defender_GetTopic	defender.c	SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Defender_MatchTopic	defender.c	SUCCESS	0.00%	0.00%	0	0.00%	0.00%	0	0.00%	0.00%	0	0.00%	0.00%	0	0.00%	0.00%	0	100.00%	100.00%	0
extractThingNameLength	defender.c	SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
getTopicLength	defender.c	SUCCESS	93.33%	90.48%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
matchApi	defender.c	SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
matchBridge	defender.c	SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
matchPrefix	defender.c	SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Table 8: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for the open-source code as the system is running with RAG on text-embedding-ada-002 Embedding Model while the CBMC is using MiniSAT Solver

4.6 Performance with text-embedding-ada-002 for OpenAI and CaDiCal Solver

For the single file bubble sort example, the RAG approach with text-embedding-ada-002 model and CaDiCal Solver had the following performance

Total execution time: 61.51 seconds
Average harness generation time: 4.01 seconds
Average verification time: 0.33 seconds
Average evaluation time: 0.37 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
bubble_sort		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
create_array		SUCCESS	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
merge_sorted_arrays		SUCCESS	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-
filter_array		SUCCESS	100.00%	100.00%	4	100.00%	100.00%	0	-	-	-	-	-	-
find_median		FAILED	100.00%	100.00%	1	100.00%	100.00%	1	94.74%	85.71%	1	100.00%	100.00%	1

Table 9: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness a single file code as the system is running with RAG on text-embedding-ada-002 Embedding Model while the CBMC is using CaDiCal Solver

For the large open-source code example, the RAG approach with text-embedding-ada-002 model embedding model and CaDiCal Solver had the following performance

Total execution time: 329.43 seconds
Average harness generation time: 5.29 seconds
Average verification time: 0.36 seconds
Average evaluation time: 0.33 seconds

Function	File	Status	Version 1			Version 2			Version 3			Version 4			Version 5			Version 6		
			Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors	Total %	Func %	Errors
Defender_GetTopic	defender.c	FAILED	95.00%	91.30%	1	95.24%	91.30%	1	96.00%	91.30%	1	95.56%	91.30%	1	96.00%	91.30%	1	91.49%	82.61%	1
Defender_MatchTopic	defender.c	SUCCESS	97.18%	95.74%	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
extractThingNameLength	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-
getTopicLength	defender.c	SUCCESS	93.10%	90.48%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-
matchApi	defender.c	FAILED	93.33%	100.00%	1	94.03%	100.00%	1	94.03%	100.00%	1	93.10%	100.00%	1	93.10%	100.00%	1	100.00%	100.00%	1
matchBridge	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-	-	-	-
matchPrefix	defender.c	SUCCESS	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	1	100.00%	100.00%	0	-	-	-	-	-	-

Table 10: Table featuring Total Coverage, Functional Coverage and Total Error in each version of Generated Harness for the open-source code as the system is running with RAG on text-embedding-ada-002 Embedding Model while the CBMC is using CaDiCal Solver

5. Refinement Observation

Let's analyze the performance metrics of different versions of the code to identify areas for improvement which resulted in improved coverage and reduced errors.

5.1 Changes to Reduce the Number of Errors

For this study, let us consider the case 4.4 for the bubble sort single file and the open-source code. In the single file case, as shown in Table 5, the function filter_array initially encountered four errors due to incorrect null pointer dereferencing, which were successfully reduced to zero in the subsequent iteration. As illustrated in Figure 4, version 2 on the right correctly dereferences the variables, unlike version 1, which simply calls the target functions. In the open-source code case, as shown in Table 6, the function matchApi had initially encountered one error due to macro redefinition, which was resolved in the subsequent iteration. As illustrated in Figure 5, version 4 on the right removes one of the header files which contains the macros to resolve the redefinition issue.

v1.c

results > openai > 20250502_120546 > harnesses > filter_array > C v1.c > ...

```c
#include <stdlib.h>  // Standard library

extern int* filter_array(int* arr, int size, int threshold, int* result_size);

extern int nondet_int(void);

void main() {
    int size = nondet_int();
    __CPROVER_assume(size >= -1 && size <= 8); // reasonable small bound for coverage

    int threshold = nondet_int();

    // Allocate arr or use NULL nondeterministically
    int arr_is_null = nondet_int();
    int* arr = NULL;
    if (arr_is_null == 0 && size > 0) {
        arr = (int*)malloc(sizeof(int) * size);
        __CPROVER_assume(arr != NULL);
        for (int i = 0; i < size; ++i) {
            arr[i] = nondet_int();
        }
    }

    // Allocate result_size or use NULL nondeterministically
    int rs_is_null = nondet_int();
    int* result_size = NULL;
    if (rs_is_null == 0) {
        result_size = (int*)malloc(sizeof(int));
        __CPROVER_assume(result_size != NULL);
    }

    int* result = filter_array(arr, size, threshold, result_size);

    // If result was allocated, free it
    if (result != NULL) {
        free(result);
    }
    if (arr != NULL) {
        free(arr);
    }
    if (result_size != NULL) {
        free(result_size);
    }
}
```

v2.c

results > openai > 20250502_120546 > harnesses > filter_array > C v2.c > ...

```c
#include <stdlib.h>  // Standard library

extern int* filter_array(int* arr, int size, int threshold, int* result_size);
extern int nondet_int(void);

void main() {
    int size = nondet_int();
    __CPROVER_assume(size >= -1 && size <= 8);

    int threshold = nondet_int();

    // Allocate arr or use NULL nondeterministically
    int arr_is_null = nondet_int();
    int* arr = NULL;
    if (arr_is_null == 0 && size > 0) {
        arr = (int*)malloc(sizeof(int) * size);
        __CPROVER_assume(arr != NULL);
        for (int i = 0; i < size; ++i) {
            arr[i] = nondet_int();
        }
    }

    // Allocate result_size or use NULL nondeterministically
    int rs_is_null = nondet_int();
    int* result_size = NULL;
    if (rs_is_null == 0) {
        result_size = (int*)malloc(sizeof(int));
        __CPROVER_assume(result_size != NULL);
    }

    // Prevent null pointer dereference: only call if result_size != NULL
    int* result = NULL;
    if (result_size != NULL) {
        result = filter_array(arr, size, threshold, result_size);
    }

    // Assert: if result_size is not NULL, it must not be dereferenced if arr is NULL or size <= 0
    if (result_size != NULL && (arr == NULL || size <= 0)) {
        // The function sets *result_size = 0 in this case
        __CPROVER_assert(*result_size == 0, "result_size should be 0 if input is invalid");
    }

    if (result != NULL) {
        free(result);
    }
    if (arr != NULL) {
        free(arr);
    }
    if (result_size != NULL) {
        free(result_size);
    }
}
```

Figure 4: Comparison of Different Versions of Generated Harness for filter_array function

v3.c

results > openai > 20250502_115824 > harnesses > defender.c:matchApi > C v3.c > ...

```c
#include "defender_config_defaults.h"
#include "defender.h"
#include <stdlib.h>  // Standard library
#include <string.h>  // Standard library

/* Nondet function declarations for CBMC */
extern unsigned int nondet_uint(void);
extern char nondet_char(void);

/* Extern declaration for the function under test */
extern DefenderStatus_t matchApi(const char * pRemainingTopic,
                                 uint16_t remainingTopicLength,
                                 DefenderTopic_t * pOutApi);

void main()
{
    /* --- Minimal, stack-based harness to avoid macro redefinition errors --- */

    /* Compute the maximum topic length from project macros */
    uint16_t max_json_accepted = DEFENDER_API_LENGTH_JSON_FORMAT + DEFENDER_API_LENGTH_ACCEPTED_SUFFIX;
    uint16_t max_json_rejected = DEFENDER_API_LENGTH_JSON_FORMAT + DEFENDER_API_LENGTH_REJECTED_SUFFIX;
    uint16_t max_cbor_accepted = DEFENDER_API_LENGTH_CBOR_FORMAT + DEFENDER_API_LENGTH_ACCEPTED_SUFFIX;
    uint16_t max_cbor_rejected = DEFENDER_API_LENGTH_CBOR_FORMAT + DEFENDER_API_LENGTH_REJECTED_SUFFIX;

    uint16_t max_topic_length = max_json_accepted;
    if (max_json_rejected > max_topic_length) max_topic_length = max_json_rejected;
    if (max_cbor_accepted > max_topic_length) max_topic_length = max_cbor_accepted;
    if (max_cbor_rejected > max_topic_length) max_topic_length = max_cbor_rejected;

    /* Nondet input, constrained to valid topic length range */
    uint16_t remainingTopicLength = (uint16_t)nondet_uint();
    __CPROVER_assume(remainingTopicLength > 0);
    __CPROVER_assume(remainingTopicLength <= max_topic_length);

    /* Use stack allocation to avoid malloc/free and macro redefinition errors */
    char topic_buf[256];
    __CPROVER_assume(remainingTopicLength < sizeof(topic_buf));
    for (uint16_t i = 0; i < remainingTopicLength; i++)
    {
        topic_buf[i] = nondet_char();
    }
    /* Defensive null-termination for strncmp safety (not required by function, but safe) */
    topic_buf[remainingTopicLength] = '\0';

    DefenderTopic_t outApi;

    /* Call the function under test */
    DefenderStatus_t result = matchApi(topic_buf, remainingTopicLength, &outApi);

    /* Assert result is within valid enum range (DefenderNoMatch or DefenderSuccess) */
    __CPROVER_assert(
        result == DefenderNoMatch || result == DefenderSuccess,
        "Result should be DefenderNoMatch or DefenderSuccess"
    );
}
```

v4.c

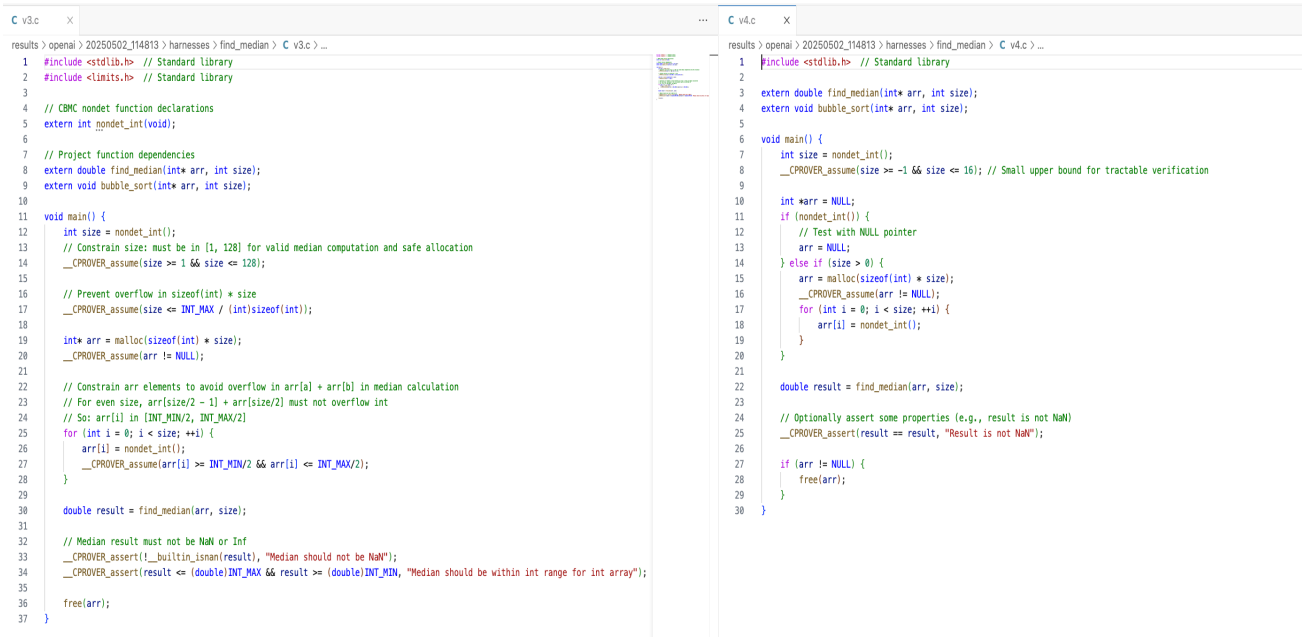
results > openai > 20250502_115824 > harnesses > defender.c:matchApi > C v4.c > ...

```c
#include "defender.h"
#include <stdlib.h>  // Standard library
#include <string.h>  // Standard library

/* Nondet function declarations for CBMC */
extern unsigned int nondet_uint(void);
extern char nondet_char(void);

/* Extern declaration for the function under test */
extern DefenderStatus_t matchApi(const char * pRemainingTopic,
                                 uint16_t remainingTopicLength,
                                 DefenderTopic_t * pOutApi);

void main()
{
    /* --- Minimal harness to avoid macro redefinition errors and preprocessing issues --- */

    /* Compute the maximum topic length using project macros */
    uint16_t max_json_accepted = DEFENDER_API_LENGTH_JSON_FORMAT + DEFENDER_API_LENGTH_ACCEPTED_SUFFIX;
    uint16_t max_json_rejected = DEFENDER_API_LENGTH_JSON_FORMAT + DEFENDER_API_LENGTH_REJECTED_SUFFIX;
    uint16_t max_cbor_accepted = DEFENDER_API_LENGTH_CBOR_FORMAT + DEFENDER_API_LENGTH_ACCEPTED_SUFFIX;
    uint16_t max_cbor_rejected = DEFENDER_API_LENGTH_CBOR_FORMAT + DEFENDER_API_LENGTH_REJECTED_SUFFIX;

    uint16_t max_topic_length = max_json_accepted;
    if (max_json_rejected > max_topic_length) max_topic_length = max_json_rejected;
    if (max_cbor_accepted > max_topic_length) max_topic_length = max_cbor_accepted;
    if (max_cbor_rejected > max_topic_length) max_topic_length = max_cbor_rejected;

    /* Nondet input, constrained to valid topic length range */
    uint16_t remainingTopicLength = (uint16_t)nondet_uint();
    __CPROVER_assume(remainingTopicLength > 0);
    __CPROVER_assume(remainingTopicLength <= max_topic_length);

    /* Allocate buffer with malloc to avoid macro redefinition issues from header includes */
    char * topic_buf = (char *)malloc(remainingTopicLength + 1);
    __CPROVER_assume(topic_buf != NULL);

    for (uint16_t i = 0; i < remainingTopicLength; i++)
    {
        topic_buf[i] = nondet_char();
    }
    /* Defensive null-termination for strncmp safety (not required by function, but safe for CBMC) */
    topic_buf[remainingTopicLength] = '\0';

    DefenderTopic_t outApi;

    /* Call the function under test */
    DefenderStatus_t result = matchApi(topic_buf, remainingTopicLength, &outApi);

    /* Assert result is within valid enum range (DefenderNoMatch or DefenderSuccess) */
    __CPROVER_assert(
        result == DefenderNoMatch || result == DefenderSuccess,
        "Result should be DefenderNoMatch or DefenderSuccess"
    );

    /* Free allocated memory to avoid CBMC memory-leak errors */
    free(topic_buf);
}
```

Figure 5: Comparison of Different Versions of Generated Harness for matchApi Function

5.2 Changes to Improve the Coverage

For this study, let us consider case 4.6 for bubble sort single file and the open-source code. In both cases, the coverages improves while number of errors stays the same. In the single file case, as shown in Table 9, the coverage for the find_median function increases from 94.75% to 100%. From Figure 6, the version 4 on the right has better CBMC coverage because it constrains array elements to avoid overflow, adds more thorough assertions on the output, and ensures all code paths (including memory management and result checks) are exercised more robustly. In the open-source code case, as shown in Table 10, the coverage for the getTopicLength function increases from 93% to 100%. From Figure 7, the version 2 on the right has better CBMC coverage because it uses more precise constraints and assumptions to ensure all relevant input ranges and edge cases are explored, avoids undefined behavior (like dereferencing NULL), adds checks for output validity (e.g., not NaN, within range), and constrains array elements to prevent overflows-allowing CBMC to systematically and exhaustively verify more program behaviors and branches compared to the left version, which may miss cases or terminate early due to invalid inputs or unconstrained states.



v3.c

results › openai › 20250502_114813 › harnesses › find_median › C v3.c › ...

```c
#include <stdlib.h>  // Standard library
#include <limits.h>  // Standard library

// CBMC nondet function declarations
extern int nondet_int(void);

// Project function dependencies
extern double find_median(int* arr, int size);
extern void bubble_sort(int* arr, int size);

void main() {
    int size = nondet_int();
    // Constrain size: must be in [1, 128] for valid median computation and safe allocation
    __CPROVER_assume(size >= 1 && size <= 128);

    // Prevent overflow in sizeof(int) * size
    __CPROVER_assume(size <= INT_MAX / (int)sizeof(int));

    int* arr = malloc(sizeof(int) * size);
    __CPROVER_assume(arr != NULL);

    // Constrain arr elements to avoid overflow in arr[a] + arr[b] in median calculation
    // For even size, arr[size/2 - 1] + arr[size/2] must not overflow int
    // So: arr[i] in [INT_MIN/2, INT_MAX/2]
    for (int i = 0; i < size; ++i) {
        arr[i] = nondet_int();
        __CPROVER_assume(arr[i] >= INT_MIN/2 && arr[i] <= INT_MAX/2);
    }

    double result = find_median(arr, size);

    // Median result must not be NaN or Inf
    __CPROVER_assert(!__builtin_isnan(result), "Median should not be NaN");
    __CPROVER_assert(result <= (double)INT_MAX && result >= (double)INT_MIN, "Median should be within int range for int array");

    free(arr);
}
```

v4.c

results › openai › 20250502_114813 › harnesses › find_median › C v4.c › ...

```c
#include <stdlib.h>  // Standard library

extern double find_median(int* arr, int size);
extern void bubble_sort(int* arr, int size);

void main() {
    int size = nondet_int();
    __CPROVER_assume(size >= -1 && size <= 16); // Small upper bound for tractable verification

    int *arr = NULL;
    if (nondet_int()) {
        // Test with NULL pointer
        arr = NULL;
    } else if (size > 0) {
        arr = malloc(sizeof(int) * size);
        __CPROVER_assume(arr != NULL);
        for (int i = 0; i < size; ++i) {
            arr[i] = nondet_int();
        }
    }

    double result = find_median(arr, size);

    // Optionally assert some properties (e.g., result is not NaN)
    __CPROVER_assert(result == result, "Result is not NaN");

    if (arr != NULL) {
        free(arr);
    }
}
```

Figure 6: Comparison of Different Versions of Generated Harness for find_median Function

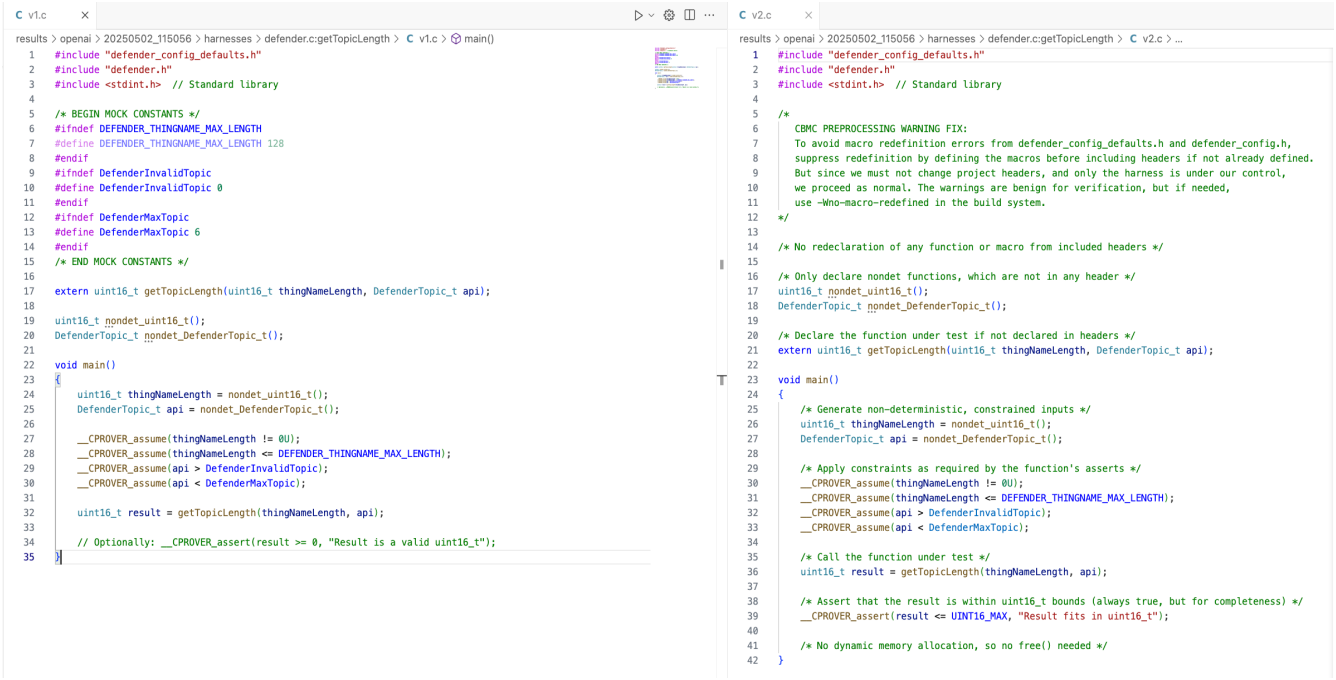


Figure 7: Comparison of Different Versions of Generated Harness for getTopicLength Function

6. Conclusion and Future Direction

The current version of CBMC Harness Generation System can generate verification harness functions for functions that involve memory and arithmetic operations. While the system produces error-free and fully covered harnesses for most memory-based functions within a specified number of refinements, some arithmetic-based functions still encounter errors, necessitating additional prompts to address these issues. Furthermore, this study can be extended to explore prompt engineering techniques applied in the generator and evaluator to enhance the quality of the generated harness function which includes performance comparison between zero-shot prompts and one-shot prompts. In terms of the system's application to generalized source code, it currently supports single-file code and FreeRTOS-based directories. Future work can focus on making the system more generalizable across various codebases.